CloudSecOps

Architecture Cloud Native Sécurisée AWS

1. Introduction **3**
1.1 Contexte **3**
1.2 Objectifs du projet 3
1.3 Périmètre 3
2. Cartographie du Système d'Information **4**
2.1 Cartographie fonctionnelle 4
2.2 Cartographie applicative 4
2.3 Cartographie technique 5
3. Architecture globale **5**
3.1 Architecture logique 5
3.2 Principes d'architecture **5**
4. Spécifications fonctionnelles **6**
4.1 Exigences fonctionnelles 6
4.2 Exigences non fonctionnelles 6
5. Modélisation UML **6**
5.1 Diagramme de classes 6
6. Infrastructure as Code **7**
6.1 Organisation Terraform 7
6.2 Gestion du state 8
7. DevSecOps **8**
8. Sécurité & Conformité **8**
9. Gestion des incidents **8**
9.1 Scénario 8
9.2 Analyse 9
10. Budget prévisionnel **9**
10.1 Budget AWS 9
10.2 Budget outils 9
11. Gestion de projet **9**
11.1 Méthodologie Agile 9
11.2 Gestion via GitHub 10
12. Critères de réussite **10**
Conclusion **11**

1. Introduction

1.1 Contexte

La transformation numérique des organisations s'accompagne d'une migration massive vers des environnements Cloud publics, notamment Amazon Web Services (AWS).

Cependant, cette transition introduit de nouveaux risques :

- Mauvaises configurations IAM
- Exposition involontaire de services (S3, bases de données)
- Faible intégration de la sécurité dans les pipelines CI/CD
- Manque de supervision et de traçabilité des actions

Les incidents récents démontrent que les erreurs de configuration constituent une cause majeure de compromission.
Dans ce contexte, la sécurisation d'une infrastructure Cloud ne peut plus être envisagée comme une couche additionnelle mais doit être intégrée dès la conception selon une approche Security by Design et DevSecOps.

1.2 Objectifs du projet

Le projet vise à :

- Concevoir une architecture Cloud AWS sécurisée
- Automatiser l'infrastructure via Terraform (Infrastructure as Code)
- Intégrer la sécurité dans le pipeline CI/CD
- Mettre en œuvre une politique Zero Trust
- Simuler un incident de sécurité
- Réaliser une analyse forensique
- Produire une documentation GRC conforme aux bonnes pratiques ISO 27001

1.3 Périmètre

Inclus

- Infrastructure AWS (VPC, EC2, RDS, S3, IAM)
- Application Web sécurisée
- Pipeline DevSecOps
- Journalisation & supervision
- Simulation d'attaque
- Analyse forensique
- Documentation complète

Exclus

- Multi-région haute disponibilité
- SIEM externe avancé
- Interface front-end complexe

2. Cartographie du Système d'Information

2.1 Cartographie fonctionnelle

Acteurs
Utilisateur final
Administrateur
DevOps
Analyste sécurité
Flux principaux
Utilisateur → HTTPS → API
API → Base de données (RDS)
API → S3
GitHub → Pipeline CI/CD → AWS
AWS → CloudTrail → Logs → Analyse

2.2 Cartographie applicative

Composant	Rôle
Backend API	Gestion logique métier
PostgreSQL RDS	Stockage données
S3	Stockage objets
Terraform	Provisionnement
GitHub Actions	CI/CD
CloudTrail	Traçabilité
CloudWatch	Monitoring

2.3 Cartographie technique

Architecture réseau :

VPC isolé
Subnets publics
Subnets privés
Security Groups restrictifs
IAM Least Privilege
Bastion (optionnel)
Logs centralisés

3. Architecture globale

3.1 Architecture logique

Internet
↓
AWS WAF
↓
EC2 (API Dockerisée)
↓
RDS (Subnet privé)
↓
S3 (chiffré)
↓
CloudTrail & CloudWatch

3.2 Principes d'architecture

Séparation réseau stricte
Chiffrement au repos (AES-256)
Chiffrement en transit (TLS 1.2+)
Moindre privilège IAM
Journalisation exhaustive
Infrastructure as Code

4. Spécifications fonctionnelles

4.1 Exigences fonctionnelles

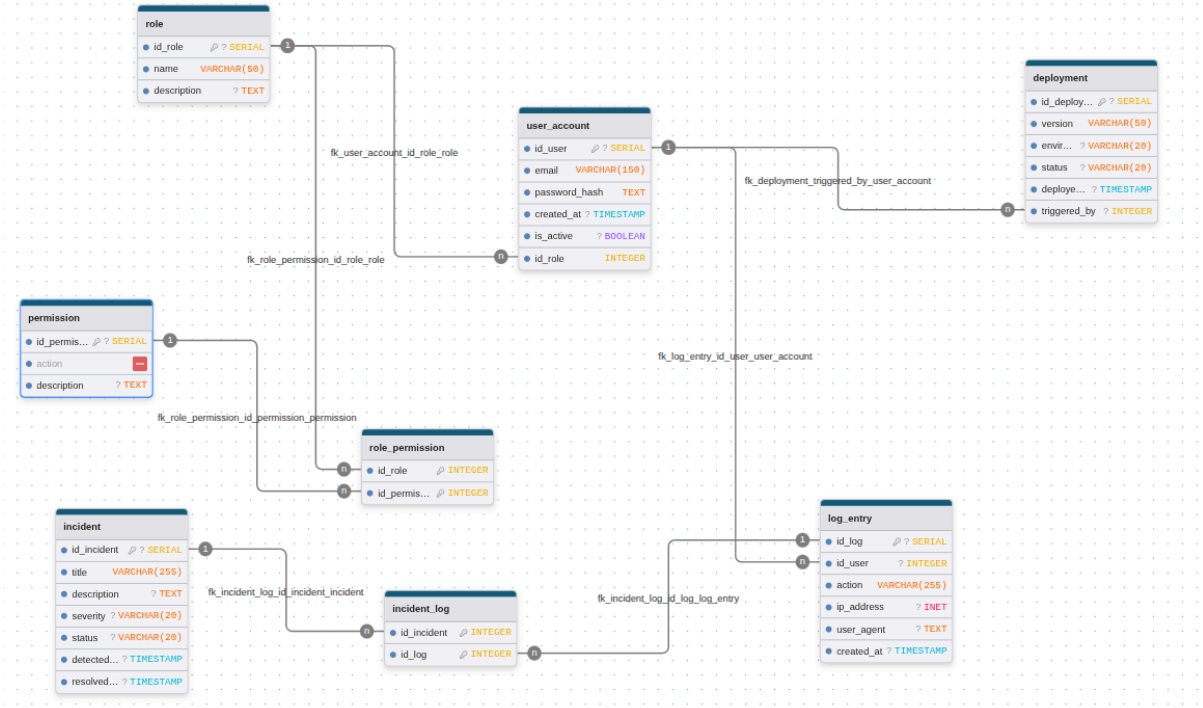
ID	Description	Priorité
F01	Authentification JWT	Haute
F02	Gestion RBAC	Haute
F03	Journalisation actions	Haute
F04	Déploiement Terraform automatisé	Haute
F05	Détection d’incident	Haute
F06	Génération rapport forensique	Moyenne

4.2 Exigences non fonctionnelles

Disponibilité cible : 99%
Temps réponse API < 200 ms
Logs horodatés
Chiffrement systématique
Séparation environnements dev / prod

5. Modélisation UML

5.1 Diagramme de classes



5.2 Diagramme de séquence

Séquence Authentification

Utilisateur
→ API
→ DB
→ Vérification
→ Génération JWT
→ Retour token

Séquence Déploiement CI/CD

Developer
→ GitHub
→ GitHub Actions
→ Scan sécurité
→ Build Docker
→ Terraform apply
→ AWS

Séquence Incident

Attaque
→ CloudTrail
→ CloudWatch
→ Détection anomalie
→ Notification
→ Analyse logs
→ Rapport

6. Infrastructure as Code

6.1 Organisation Terraform

```
/terraform
  /modules
    /vpc
    /ec2
    /rds
    /s3
    /iam
```

6.2 Gestion du state

Backend local (phase initiale)
Option S3 chiffré

7. DevSecOps

Pipeline CI/CD intégrant :

- SAST
- Scan dépendances
- Scan Docker
- DAST
- Quality gates

Objectif : détecter vulnérabilités avant déploiement.

8. Sécurité & Conformité

- Référentiel ISO 27001
- CIS AWS Benchmark
- OWASP ASVS
- ANSSI recommandations cloud
- Démarche Zero Trust

9. Gestion des incidents

9.1 Scénario

- Compromission IAM simulée
- Tentative accès S3
- Journalisation CloudTrail

9.2 Analyse

- Reconstruction timeline
- Identification vecteur
- Mesures correctives

10. Budget prévisionnel

10.1 Budget AWS

Ressource	Coût
EC2 t3.micro	Free Tier
RDS micro	Free Tier
S3 (≤5 Go)	Free Tier
Logs	Faible

Budget cible : 0€ (Free Tier)

10.2 Budget outils

Tous les outils sont open source ou gratuits :

- Terraform
- GitHub
- Docker

- OWASP ZAP
- Trivy

11. Gestion de projet

11.1 Méthodologie Agile

Sprint	Période	Objectif
Sprint 0	Semaine 1–2	Cadrage & Setup
Sprint 1	Semaine 3–4	Infrastructure Base
Sprint 2	Semaine 5–6	Infrastructure Applicative
Sprint 3	Semaine 7–8	Backend
Sprint 4	Semaine 9–10	DevSecOps
Sprint 5	Semaine 11–12	Sécurité & Incident

11.2 Gestion via GitHub

Outils utilisés
GitHub Issues (gestion tâches)
Milestones (sprints)
Labels (infra, dev, security, doc)
Pull Requests (revue)
Git tags (versions)
Project Board (Kanban)
Traçabilité

Chaque exigence fonctionnelle = Issue
Chaque sprint = Milestone
Chaque livrable = Release

12. Critères de réussite

Infrastructure automatisée
Pipeline sécurisé opérationnel
Incident détecté
Rapport forensique produit

Documentation complète

Conclusion

Ce cahier des charges formalise la conception d'une plateforme Cloud sécurisée intégrant Infrastructure as Code, DevSecOps et gestion d'incident.
Il constitue une base méthodologique et technique alignée avec les exigences d'un projet d'ingénierie en cybersécurité.